

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	GAUTHAM G	Reg. No.:	192524371
Section / Batch:	SLOT-B	Date:	28/08/2026

Code of Conduct

I am GAUTHAM G (192524371) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student : GAUTHAM G

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on:

(20 Marks)

- Clustering effect
- Memory usage
- Performance under high load factor
- Which method is more suitable for large-scale applications and why?

2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays?

(20 Marks)

3. Compare Quadratic Probing and Double Hashing in terms of:

(20 Marks)

- Clustering issues
- Collision resolution efficiency
- Suitability for dense hash tables

4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following:

(20 Marks)

- Practical significance
- Impact on algorithm selection

5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)

- Space complexity (stack usage)
- Ease of implementation
- Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) Linear Probing Vs Separate Chaining. ①

Feature	Linear Probing	Separate Chaining.
Clustering.	Has primary clustering.	NO primary clustering.
Memory.	Uses less Memory.	Uses extra memory for linked lists.
High load Factor.	Performance decreases quickly.	Performs better.
Implement.	Simple	Slightly Complex
Large-Scale Use	Less suitable for very high load	More Suitable.

a) Clustering Effect:

* Linear probing creates primary clustering because collided elements are stored in consecutive positions:

* Separate chaining stores collided elements in a linked list, so clustering is avoided.

b) Memory Usage:

* Linear probing needs only the hash table, so it uses less memory. Separate chaining needs extra memory for linked-list nodes and pointers.

c) High Load Factor:

- * Linear probing becomes slow when the table is nearly full.

- * Separate chaining can handle a higher load factor more efficiently.

d) Large -scale Application:

- * Separate chaining is generally better for large-scale application because it handles collisions and high load factors better and supports dynamic insertion and deletion.

2) Quick Sort vs Merge Sort For Linked List.

Feature	Quick Sort	Merge Sort
Best / Avg time	$O(n \log n)$	$O(n \log n)$
Worst time	$O(n^2)$	$O(n \log n)$
Extra Space	$O(\log n)$ average	$O(\log n)$ stack
Linked-List Suitability	Moderate	Excellent
Stability	Usually not stable	stable

Linked List :

(2)

For linked list of 10 million nodes, Merge sort is preferable because :

- i) It gives guaranteed $O(n \log n)$ time.
- ii) It does not require random access.
- iii) It works efficiently with linked-list pointers.
- iv) It has predictable performance.
- v) It can merge lists without copying all elements.

Arrays :

* For arrays, Quicksort is often preferred because it can work in-place and has good cache performance.

Conclusion :

* Linked List $\rightarrow$ Merge Sort.

* Array $\rightarrow$ Quick Sort is often preferred.

3)

Clustering Issues :

a)

* Quadratic probing reduces primary clustering but can have secondary clustering.

* Double hashing greatly reduces both types of clustering because it uses a second hash function.

b) Collision Resolution:

- * Quadratic probing checks positions using a quadratic formula

- * Double hashing uses a second hash function to decide the next position, giving better distribution

- * Therefore, double hashing generally provides better collision resolution

c) Dense Hash Tables:

- * When the table becomes full, collisions increase.

- * Double hashing is more suitable for dense hash tables because it distributes elements more evenly

Quadratic Probing Vs Double Hashing:

Feature	Quadratic Probing	Double Hashing.
Primary clustering	Avoids	Avoids.
Secondary clustering	Can occur	very low
Collision handling	Good	Better
Dense table	Less suitable	More suitable
complexity	$O(1)$ average	$O(1)$ average

4) Best - Case Vs Worst - Case Sorting Complexity ③

Algorithm	Best Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$

a) Practical Significance:

* Best - case shows the minimum time required for favourable input.

* Worst - case show the maximum time required for unfavourable input.

* Worst - case complexity is important when dealing with large datasets because poor performance can make the program very slow.

b) Impact On Algorithm Selection:

* Small or nearly sorted data $\rightarrow$ Insertion Sort.

* Large data $\rightarrow$ Merge Sort / Heap Sort.

* General - purpose array sorting $\rightarrow$ Quick Sort.

* Guaranteed $O(n \log n)$ performance $\rightarrow$ Merge Sort or Heap Sort.

Conclusion: Algorithm selection should consider both best-case and worst-case performance

5) Recursive vs Iterative Quick Sort:

Feature	Recursive	Iterative
Implement	Easy	More Complex
Stack Usage	Uses system stack	Uses explicit stack
Average Space	$O(\log n)$	$O(\log n)$
Worst-Case Space	$O(n)$	can be $O(n)$
Stack Overflow	Higher Risk	Lower Risk
Understanding	Easier	Slightly Difficult

a) Space Complexity:

* Recursive Quick Sort uses the function call stack. * Average: $O(\log n)$ * Worst case: $O(n)$

* Iterative Quick Sort uses an explicit stack, giving better control over memory.

b) Ease Of Implementation :

(L)

- * Recursive Quick sort is simpler and easier to understand.

- * Iterative Quick sort requires manually creating and managing a stack.

c) Stack Overflow :

- * Recursive Quick sort can cause stack overflow when partitions become highly unbalanced.

- * Iterative Quick sort has lower risk of call-stack overflow because it uses an explicit stack.

Conclusion :

- * Recursive Quick sort → Simple and easy to implement

- * Iterative Quick sort → Better control over stack memory and lower call-stack overflow risk.

Wgong